CSE STUDY HUB

C++ Programming

UNIT 4 · Object-Oriented Design Patterns

What this unit is about

This unit builds understanding of **Object-Oriented Design Patterns**. These notes are designed for quick learning and exam revision: learn the meaning, understand the working, practise an example, and finish with comparisons and key terms.

Syllabus coverage checklist

1. Design patterns: reusable solutions to recurring design problems

2. Creational patterns: Singleton, Factory, Builder and Prototype

3. Structural patterns: Adapter, Decorator and Facade

4. Behavioural patterns: Strategy, Observer and State

5. Class relationships, composition vs inheritance

6. Choosing a pattern; benefits, trade-offs and overengineering risks

7. Applying patterns to small C++ design scenarios

How to study: Read each topic once for meaning, underline definitions and formulas, reproduce diagrams/steps without looking, then answer the revision questions on Page 5.

Exam answer pattern

For a 5-mark answer: (1) definition, (2) labelled diagram or formula, (3) working/components in steps, (4) example/application, and (5) one advantage, limitation or comparison.

UNIT 4 NOTES · PART 1

Design patterns: reusable solutions to recurring design problems

Design patterns: reusable solutions to recurring design problems is a syllabus concept in C++ Programming. Learn its definition, purpose, key terms and working principle. Organise the explanation into components or steps, then add a practical example, one advantage, one limitation and a comparison with a related concept.

Write in your own words: Define the concept, list its important parts/steps, and attach one example from a system, program, workplace or numerical problem.

Creational patterns: Singleton, Factory, Builder and Prototype

For Creational patterns: Singleton, Factory, Builder and Prototype, identify the given values and the quantity to find before selecting a formula. Keep units consistent, show substitutions, calculate carefully and perform a reasonableness check. Write the formula and one solved example in your revision copy.

Write in your own words: Define the concept, list its important parts/steps, and attach one example from a system, program, workplace or numerical problem.

Structural patterns: Adapter, Decorator and Facade

Structural patterns: Adapter, Decorator and Facade is a syllabus concept in C++ Programming. Learn its definition, purpose, key terms and working principle. Organise the explanation into components or steps, then add a practical example, one advantage, one limitation and a comparison with a related concept.

Write in your own words: Define the concept, list its important parts/steps, and attach one example from a system, program, workplace or numerical problem.

Connect and apply

For a practical question on Object-Oriented Design Patterns, first identify which syllabus concept is being tested. State assumptions, draw the relevant structure/flow, work through the steps in order, and finish by interpreting the result. Mention complexity, reliability, accuracy, cost or ethical impact whenever it is relevant.

UNIT 4 NOTES · PART 2

Behavioural patterns: Strategy, Observer and State

For Behavioural patterns: Strategy, Observer and State, identify the given values and the quantity to find before selecting a formula. Keep units consistent, show substitutions, calculate carefully and perform a reasonableness check. Write the formula and one solved example in your revision copy.

Write in your own words: Define the concept, list its important parts/steps, and attach one example from a system, program, workplace or numerical problem.

Class relationships, composition vs inheritance

The OSI model has Physical, Data Link, Network, Transport, Session, Presentation and Application layers. Each layer provides services to the one above and uses the one below; encapsulation adds headers as data moves down.

Write in your own words: Define the concept, list its important parts/steps, and attach one example from a system, program, workplace or numerical problem.

Connect and apply

For a practical question on Object-Oriented Design Patterns, first identify which syllabus concept is being tested. State assumptions, draw the relevant structure/flow, work through the steps in order, and finish by interpreting the result. Mention complexity, reliability, accuracy, cost or ethical impact whenever it is relevant.

UNIT 4 NOTES · PART 3

Choosing a pattern; benefits, trade-offs and overengineering risks

The OSI model has Physical, Data Link, Network, Transport, Session, Presentation and Application layers. Each layer provides services to the one above and uses the one below; encapsulation adds headers as data moves down.

Write in your own words: Define the concept, list its important parts/steps, and attach one example from a system, program, workplace or numerical problem.

Applying patterns to small C++ design scenarios

Applying patterns to small C++ design scenarios is a syllabus concept in C++ Programming. Learn its definition, purpose, key terms and working principle. Organise the explanation into components or steps, then add a practical example, one advantage, one limitation and a comparison with a related concept.

Write in your own words: Define the concept, list its important parts/steps, and attach one example from a system, program, workplace or numerical problem.

Connect and apply

For a practical question on Object-Oriented Design Patterns, first identify which syllabus concept is being tested. State assumptions, draw the relevant structure/flow, work through the steps in order, and finish by interpreting the result. Mention complexity, reliability, accuracy, cost or ethical impact whenever it is relevant.

QUICK REVISION & EXAM PRACTICE

Key takeaways

• **Design patterns: reusable solutions to recurring design problems** — know the definition, core working and one example.

• **Creational patterns: Singleton, Factory, Builder and Prototype** — know the definition, core working and one example.

• **Structural patterns: Adapter, Decorator and Facade** — know the definition, core working and one example.

• **Behavioural patterns: Strategy, Observer and State** — know the definition, core working and one example.

• **Class relationships, composition vs inheritance** — know the definition, core working and one example.

• **Choosing a pattern; benefits, trade-offs and overengineering risks** — know the definition, core working and one example.

• **Applying patterns to small C++ design scenarios** — know the definition, core working and one example.

Worked-answer framework

Scenario: A question asks you to explain or apply a concept from Object-Oriented Design Patterns. **Step 1:** identify the relevant term and define it. **Step 2:** write the formula, components or ordered process. **Step 3:** apply it to the given data/scenario. **Step 4:** state the result and justify it. **Step 5:** mention one limitation or precaution.

Self-test questions

Q1. Define Design patterns: reusable solutions to recurring design problems and explain its importance.

Q2. Explain the working/steps involved in Creational patterns: Singleton, Factory, Builder and Prototype.

Q3. Differentiate between two related concepts from Object-Oriented Design Patterns.

Q4. Give one practical application of Applying patterns to small C++ design scenarios and discuss one limitation.

Q5. Write a structured 5-mark answer covering the main ideas of Object-Oriented Design Patterns.

Final checklist: I can define every topic · explain the process without notes · reproduce formulas/diagrams · solve one example · compare related concepts.